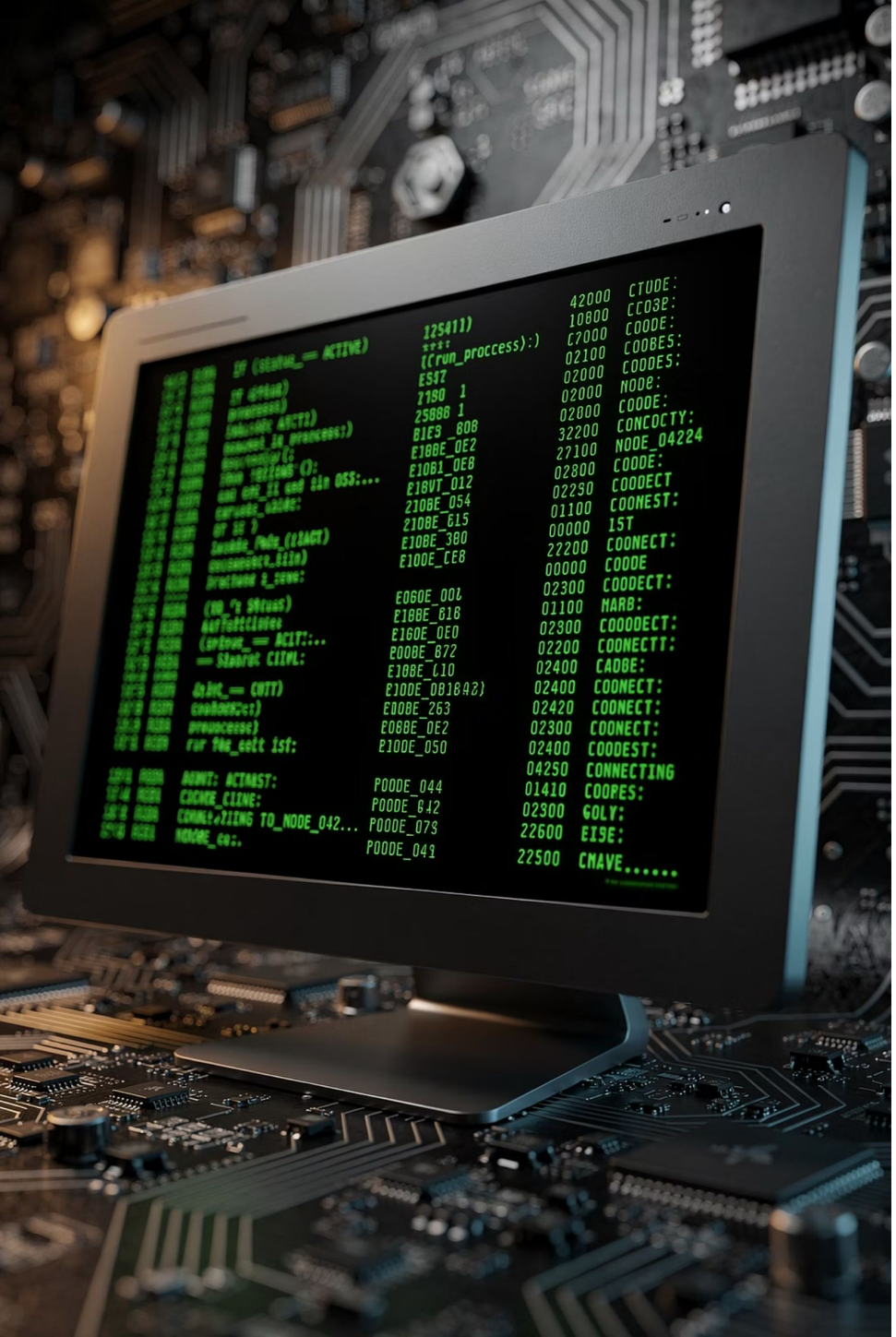


Processes & Threads — From Theory to Terminal

IIT Jodhpur · BS/BSc Programme · Semester 3 · Week 2 · Operating Systems

120-MINUTE LIVE PRACTICAL SESSION

COPY.SH/V86 · GOOGLE COLAB

What We're Doing Today

This session bridges the gap between recorded lecture theory and real, running processes on a live Linux system. Here's your 2-hour roadmap:

Phase 1 · Revise

0–50 min · Buildroot Terminal · Process states, PCB, fork/zombie

Phase 2 · Connect

50–75 min · Buildroot Terminal · IPC, pipes, shared memory

Phase 3 · Code

75–120 min · Google Colab · Threads, race conditions, locks

Pre-requisites: Watch Week 2 recorded lectures (Parts 1–3). Have a Google account ready. Open Chrome or Firefox — no local installs required.

🛠️ Tool Setup

Launching Your Linux Terminal

⚠ **copy.sh/v2 is DEAD (404).** Always use **copy.sh/v86** with Buildroot Linux. Share this direct link with students now:

0

1

Navigate

Open Chrome/Firefox → type **https://copy.sh/v86/?profile=buildroot** → press Enter. A black page with OS options appears.

0

2

Select

Click the orange text **"Buildroot Linux"** in the table. Page scrolls to the Setup panel automatically.

0

3

Launch

Scroll past all file-chooser fields → click **"Start emulation"**. Wait 20–40 seconds for boot messages to complete.

0

4

Login

At **buildroot login:** prompt → type **root** → press Enter. You see **#** prompt. You're inside a live Linux system!

Why Buildroot? It's only 4.9 MB, boots in 20–40 sec, has /proc/, ps, grep, mkfifo — everything we need. All other options (Android 54 MB, Arch 15+ MB) are too slow or missing key features.

Core Concept 1 — Process States & PCB

What Is a Process?

A **process** is a program in execution — but it's far more than running code. It's an active entity with: program code, data, heap, stack, and OS metadata (PCB). Two instances of the same program = two **separate processes** with different PIDs and memory spaces.

The 5 States

NEW

PCB created, loading from disk

READY

In RAM, waiting for CPU

RUNNING

CPU executing now

WAITING

Blocked on I/O or event

TERMINATED

Done; OS reclaims memory

🍛 Indian Analogy

IRCTC Booking

NEW=form submitted · READY=queued on server · RUNNING=payment processing · WAITING=gateway busy · TERMINATED=ticket confirmed ✓

The PCB — Your Process Passport

The **Process Control Block** lives in kernel space at **/proc/[PID]/** in Linux. It stores: state, PID, program counter, registers, memory info, and open files. Context switching = save current PCB + load next PCB. **CPU is idle during this switch — pure overhead.**

CORE CONCEPT 2

Process Creation — fork(), exec(), Zombie & Orphan

fork()

Creates an **exact duplicate** of the calling process. Parent gets child's PID (>0); child gets 0. After fork, they share no memory — each can change independently. Shell does this for every command you type.

exec()

Replaces the child's code/data/heap with a new program from disk. **PID stays the same**, everything else changes. fork-exec together = how every OS launches programs.

Zombie

Child has called exit() but parent hasn't called wait(). **No CPU, no memory** — just a PCB slot. Thousands of zombies can fill the process table and crash a server.

Orphan

Parent terminates before the child. Linux's PID 1 (**init/systemd**) automatically adopts all orphans and periodically calls wait() to clean them up.

Xerox Analogy for fork(): Original document = parent. Photocopy = child. Identical at copy time, then independent. Writing on your copy doesn't affect the original.

Core Concept 3 — Threads, IPC & Concurrency

Thread vs. Process

A **thread** is the smallest unit of execution within a process. Threads **share**: code, data, heap, open files. Each thread **owns**: its own stack, registers, thread ID, and program counter. Creating a thread is lightweight — no new memory allocation needed.

🏏 Cricket Analogy

India cricket team = Process (pitch, kit, physio). Each **player = Thread** (shares dressing room, owns batting stance + jersey number). New player (thread) reuses all shared infrastructure. New team (process) = new stadium, kit, everything.

Concurrency vs. Parallelism

Concurrency	Parallelism
One chef, 500 wedding guests. Fast switching creates the *illusion* of simultaneous. Single-core CPU. Design concept — possible anywhere.	4 chefs, 4 glasses filled *at the same moment*. Multi-core hardware required. Execution concept — truly simultaneous.

IPC — How Processes Talk

Shared Memory — Open kitchen counter. Fast, no OS middleman. Needs sync (lock).

Message Passing — Zomato app order. OS routes it. Safer, slightly slower.

Pipes — One-way speaking tube. Parent-child only (ordinary pipe).

⚡ ACTIVITY 1 | 0–20 MIN · BUILDROOT TERMINAL

Process State Quiz + Terminal Exploration

Purpose: See REAL processes in REAL states on a REAL (emulated) OS. When you spot a 'Z' in the STAT column with your own hands, the zombie concept becomes unforgettable.

1

Flash Quiz (5 min)

TA reads 5 questions aloud. Hands up or chat. Q: "Name the 5 process states." Q: "What is context switch overhead?" Q: "If 4 users run the same Python file — how many processes exist?"

2

See All Processes (5 min)

At the **#** prompt, type: `ps aux` Look at the **STAT column**: R=Running, S=Sleeping, Z=Zombie, T=Stopped. Match to the professor's 5-state diagram.

3

Inspect Your Shell (5 min)

Type: `ps -p $$ -o pid,ppid,stat,cmd`This shows YOUR shell's PID, its Parent PID, state, and command. Question: What is the PPID of your shell?

4

Count Processes (5 min)

Type: `ps aux | wc -l`Expect 20–40 processes. Discuss: "Why so many processes on this minimal system — who started them all?"

TA Tip: When students spot Z (Zombie) in ps — celebrate it! "This is the zombie your professor described. It's real. It's here. You found it!" Pre-boot Buildroot on the projector 15 min before class to avoid wait time.

⚡ ACTIVITY 2 | 20–35 MIN · BUILDROOT TERMINAL

PCB Scavenger Hunt — Exploring /proc/

Purpose: The PCB is not abstract — in Linux it physically exists at **/proc/[PID]/**. This is the single most powerful "aha moment" of the session. Every field the professor listed is a real, readable file.

Commands to Run

PCB Field (Lecture)	Command
State, PID, PPid	`cat /proc/$$/status | head -15`
Where process waits	`cat /proc/$$/wchan`
Virtual + physical memory	`cat /proc/$$/status | grep -i vm`
Open file descriptors	`ls /proc/$$/fd`
The command that started it	`cat /proc/$$/cmdline`

🗂️ Scavenger Hunt — Fill In!

What is your shell's State?

Look in /proc/$$/status → State field. Write it down.

What is the PPid?

Who is the parent of your shell? That number is the parent's PID.

How much VmSize?

Virtual memory used by your shell in kB.

How many open files?

Run: `ls /proc/$$/fd | wc -l`

Analogy: "/proc/ is like the OS's Aadhaar database — every process has its own folder with its complete ID and health record." Students who finish early: compare /proc/$$/status with /proc/1/status.

⚡ ACTIVITY 3 | 35–50 MIN · BUILDROOT TERMINAL

Process Tree, fork() Simulation & Zombie Demo

Purpose: Make process trees, fork(), and zombies VISIBLE. When students create and observe a real zombie appearing in ps output, the lecture becomes a lasting memory.

View Process Tree

Type: `ps f`Buildroot has no pstree — use ps f instead. Find PID 1 (init) at the top. Trace branches downward. Every indented entry is a child of the process above.

Create a Zombie (TA Live Demo)

`sleep 100 & CHILD_PID=$!kill $CHILD_PIDps | grep -i z`Look for **Z** in STAT column — that's your zombie. Child is dead, parent hasn't called wait().

1

2

3

Simulate fork()

`echo My PID: $$sleep 30 &ps | grep sleepcat /proc/$!/status | grep -E 'Name|Pid|PPid'`The sleep process's PPid = your shell's PID. That's parent-child!

TA Discussion Questions: "Does the zombie consume CPU?" (No.) "What if we kill the PARENT instead?" (Child becomes orphan — init adopts it.) "10,000 zombies — what happens?" (Process table fills. System unusable.) If the zombie disappears fast — explain: "Shell called wait() automatically. Correct behavior. Buggy apps don't do this."

⚡ ACTIVITY 4 | 55–75 MIN · BUILDROOT TERMINAL

IPC Simulation — Shared Memory vs. Message Passing

Purpose: IPC underlies every networked app and microservice. Use the Zomato kitchen analogy to make the two methods intuitive, then prove pipes work live in Buildroot.

🍛 Shared Memory

Open dhaba counter: Chef (Process A) puts biryani on the shared counter. Delivery boy (Process B) grabs it directly — no manager. **FAST.** But if both touch at once → race condition. Fix: a LOCK on the counter.

📱 Message Passing

Zomato app order: Customer (Process A) sends order through Zomato server (Kernel). Server routes to restaurant (Process B). Organized, synchronized. Slightly slower — every exchange goes through the OS.

🔧 Pipes — Live Demo

`echo 'Hello from Process A' | catps aux | grep ash | wc -l` ← 3 processes, 2 pipes!

`mkfifo mypipe && ls -la mypipe` ← See the 'p' in permissions = named pipe file

TA Tip: Ask students to predict output BEFORE pressing Enter. When showing 'ls -la mypipe': "See the p? That is the OS marking this as a communication channel — not a regular file." Ask: "Which IPC method does YouTube use?" (Sockets — message passing over network.)

🔀 Switch Tools — Open Google Colab Now

⚠ Action Required

Keep your Buildroot tab open. Open a **NEW browser tab** for the Python coding activities. You will not need to type Linux commands for the next 30 minutes.

Colab Setup — 60 Seconds

0

1

Navigate

Go to: **https://colab.research.google.com/**

0

2

Sign In

Log in with your Google account when prompted.

0

3

New Notebook

Click **"+ New Notebook"** (top-left).

0

4

Verify

In the first cell, type: `print('Colab ready!')` → Press **Shift+Enter**. See output → ✅ Ready.

Why Colab? Python's threading library is not available in Buildroot/Bash. Colab gives a cloud Python runtime instantly — no pip install, no local setup. Students can save work to Google Drive automatically.

⚡ ACTIVITY 5 · Part A

75–90 MIN · GOOGLE COLAB

Python Threading — Creating Your First Threads

Paste each block into a **new Colab cell** and run with **Shift+Enter**. Observe the output carefully before moving to the next block.

Code Block 1 — Single Thread

```
import threading

def task_function():
    print(f'Hello from: {threading.current_thread().name}')
    print('Thread doing work...')

my_thread = threading.Thread(
    target=task_function, name='Worker-1')
my_thread.start()
my_thread.join()
print('Main thread: Worker-1 finished.')
```

Notice: `.start()` schedules the thread. `.join()` makes main thread wait for it to finish.

Code Block 2 — 5 Zomato Delivery Agents

```
import threading, time

def deliver_order(order_number):
    agent = threading.current_thread().name
    print(f'{agent} picking up Order #{order_number}')
    time.sleep(1)
    print(f'{agent} delivered Order #{order_number} ✓')

threads = []
for i in range(1, 6):
    t = threading.Thread(
        target=deliver_order,
        args=(i,), name=f'Agent-{i}')
    threads.append(t)
    t.start()

for t in threads:
    t.join()
print('\nAll 5 orders delivered!')
```

Run this cell 3 times. Does the output order change? Yes — the OS scheduler is non-deterministic. This IS concurrency in action.

⚡ ACTIVITY 5 · Part B | 90–105 MIN · GOOGLE COLAB

Race Condition → Fix It With a Lock

This is the moment the lecture clicks. Run Block 3 twice and watch the counter give a **different wrong answer each time**. Then run Block 4 — always exactly 200,000.

Code Block 3 — BROKEN (No Lock)

```
import threading
counter = 0

def increment_counter():
    global counter
    for _ in range(100000):
        # Looks like 1 step. Actually 3:
        # READ → ADD → WRITE
        # OS can interrupt between any two!
        counter = counter + 1

t1 = threading.Thread(target=increment_counter)
t2 = threading.Thread(target=increment_counter)
t1.start(); t2.start()
t1.join(); t2.join()

print(f'Final: {counter}')
```

Run 3 times — get 3 different wrong answers. This unpredictability IS the race condition.

Code Block 4 — FIXED (With Lock)

```
import threading
counter = 0
lock = threading.Lock()

def safe_increment():
    global counter
    for _ in range(100000):
        with lock:
            # CRITICAL SECTION
            # Only ONE thread here at a time.
            # Thread-2 WAITS if Thread-1 is inside.
            counter = counter + 1

t1 = threading.Thread(target=safe_increment)
t2 = threading.Thread(target=safe_increment)
t1.start(); t2.start()
t1.join(); t2.join()

print(f'Final: {counter}')
```

Always prints 200000. Trade-off: correctness costs a little speed — threads take turns at the lock.

🚽 Lock Analogy: A single-seat exam hall toilet — only one person inside at a time. Others wait in line. Correct behavior, but creates a queue. The `with lock:` pattern auto-releases even if an exception occurs — always use this pattern over `lock.acquire()`.

EXIT TICKET

105–112 MIN · 7 MINUTES

Exit Ticket — Paste Into a New Colab Cell

Paste the comment block below into a new Colab text cell and fill in your answers. This is your proof of learning — not graded for correctness, but required as evidence of participation.

Q1 — Process State Diagram

Fill in the blanks: **New** → _____ → _____ → _____ → _____The Waiting state loops back to _____ when I/O completes.

Q2 — PCB Scavenger Hunt

Name TWO PCB fields you actually READ from `/proc/$$/status` during Activity 2. Write the exact field names and the values you saw.

Q3 — Race Condition

What counter value did YOU get in Code Block 3 (no lock)? Expected was 200,000. In one sentence — why was your answer different?

Q4 — Zombie vs. Orphan

Zombie: Child is _____, parent has NOT called _____.

Orphan: Parent has _____, child is still _____.

IN-SESSION ASSIGNMENT

112–118 MIN · DUE BEFORE WEEK 3

Week 2 Assignment — Two Tasks

Directly tied to what you did today. **Graded for honest attempt** — not for correct answers. Submit before the Week 3 session via LMS.

Task 1 · Terminal Report

Run 4 Buildroot commands and screenshot each: `ps aux`, `cat /proc/$$/status | head -15`, `ps f`, `ps aux | grep ash | wc -l`. Write 4 sentences explaining what you observed in each output.

Task 2 · Zombie + Orphan Diagrams

Draw two diagrams (hand or digital): one showing a zombie process (child exits, parent never calls wait), one showing an orphan being adopted by init (PID 1). Answer 3 written questions about why zombies are dangerous.

Submission: Tasks 1+2 → make a combined PDF/Word, file name **OS_Week2_Assignment_[RollNumber].pdf**.

Key Takeaways — Week 2

Everything covered today in one reference slide. Come back to this after the session to check your understanding.

Process vs. Program

Program = static file on disk. Process = program running with memory, state, PID, and PCB. Same program can spawn many processes — each with a unique PID and memory space.

5 States + Context Switch

New → Ready → Running → Waiting → Terminated. Context switch = save current PCB + load next PCB. CPU is idle during the switch — that's the overhead cost.

PCB in Linux is Real

Lives at `/proc/[PID]/`. You read it today. Contains: state, PID, PPid, VmSize, open file descriptors. Not an abstraction — real kernel data.

fork() + exec() + Zombie/Orphan

fork() clones. exec() replaces. wait() collects the child's exit status. Zombie = dead child, no wait(). Orphan = live child, dead parent — init adopts it.

Race Condition + Lock = Your Code Superpower

Shared variable + no sync = non-deterministic wrong answer. `with lock:` = only one thread in the critical section → always correct. Trade-off: correctness vs. speed.

TA Reference — Analogies + Troubleshooting

🏏 Quick Analogy Bank

Concept	Indian Analogy
Process vs. Program	Recipe (program) vs. Rahul cooking (process)
5 States	IRCTC booking stages
PCB	Aadhaar card — unique UID, all info stored
Context Switch	Pausing IPL at over 14, ball 3 — switching to Bollywood, resuming exactly
fork()	Xerox photocopy — identical copy, then independent
Zombie	Bollywood ghost — person dead, ghost lingers till last rites
Orphan	Government (init/PID 1) becomes legal guardian
Race Condition	Two IRCTC users booking the last seat simultaneously
Lock	Single-seat exam hall toilet — one person at a time
Parallelism	4 juice stalls at a wedding — 4 glasses filled simultaneously

🔧 Troubleshooting Cheat Sheet

Problem	Fix
copy.sh/v86 loads blank	Hard refresh: Ctrl+Shift+R
Boot takes 90+ seconds	Pre-boot on projector 15 min early
pstree not found	Use `ps f` instead — same info
cat /proc/1 → permission denied	Use `$$` instead of 1
Zombie disappears too fast	Explain: shell auto-reaps = correct behavior
mkfifo permission error	Try: `mkfifo /tmp/mypipe`
Colab NameError: counter	Add `global counter` inside function
lock.acquire() hangs	Restart runtime. Teach: always use `with lock:`
Counter always 200000 without lock	Increase to 1,000,000 iterations
Colab session disconnects	Runtime → Reconnect. Re-run last cell.



Great Work Today!

You've gone from theory to terminal — from lecture slides to a live Linux kernel. The commands you ran today are the same ones used by real Linux engineers every day.

5

Activities Completed

Terminal exploration, PCB hunt, process tree, IPC pipes, Python threading

3

Tools Mastered

copy.sh/v86 Buildroot Linux, /proc/ filesystem, Google Colab Python

7

Core Concepts

Process, PCB, fork/exec, zombie/orphan, IPC, threads, race condition + lock

Next Week Preview: CPU Scheduling Algorithms — FCFS, SJF, Round Robin. Pre-watch Week 3 recorded lectures before the next session. Submit your Week 2 Assignment via LMS before then.

IIT Jodhpur · BS/BSc Operating Systems · Semester 3 · Week 2 · Processes & Threads